Contenido

Práctica 7. Publicar Flask mediante Apache usando Reverse Proxy 2
Objetivos 2
¿Qué es un Reverse Proxy? 2
Situación actual 2
Actividad 1. Habilitar módulos necesarios 2
Actividad 2. Crear Virtual Host para Flask 3
Actividad 3. Activar el sitio 3
Actividad 4. Configurar resolución de nombres 4
Actividad 5. Mantener Flask ejecutándose 4
Actividad 6. Acceder mediante Apache 5
Actividad 7. Comprobación de puertos 5
Actividad 8. Comprobar con Nmap 6
Conclusión 7
Objetivo final 8
Paso 1. Crear un VirtualHost para Flask 8
Paso 2. Activar módulos proxy 9
Paso 3. Activar el sitio 9
Paso 4. Configurar DNS local 9
Paso 5. Verificar Reverse Proxy 10
Paso 6. Añadir HTTPS al Reverse Proxy 13
Preguntas de reflexión 14
Resultado de aprendizaje cubiertos 15

Práctica 7. Publicar Flask mediante Apache usando Reverse Proxy

Objetivos

Los RAs cubiertos serán:

- **RA3.a** Instalar y configurar servidores Apache.
- **RA3.c** Desplegar aplicaciones Python.
- **RA3.e** Introducción a arquitecturas optimizadas para producción.
- **RA2.c** HTTPS frente a HTTP.

¿Qué es un Reverse Proxy?

Un proxy inverso es un servidor que recibe las peticiones de los clientes y las reenvía a otro servidor interno.

Por ejemplo Cliente → https://flask.local → Apache → Flask

El navegador piensa que habla con Apache pero Apache realmente habla con Flask.

Situación actual

La aplicación Flask está escuchando en **http://192.168.56.111:5000**

Vamos a mantener Flask en 5000 pero los usuarios accederán por **http://flask.local**

Actividad 1. Habilitar módulos necesarios

En el servidor Ubuntu:

sudo systemctl restart apache2

Comprobamos que nuestro usuario pertenece al grupo sudo: **groups**

```
profesor@srv-daw:~$ groups
profesor adm cdrom sudo dip plugdev users lxd docker
profesor@srv-daw:~$
```

Si aparece sudo, el usuario tiene permisos administrativos.

sudo a2enmod proxy

```
profesor@srv-daw:~$ sudo a2enmod proxy
Module proxy already enabled
profesor@srv-daw:~$
```

sudo a2enmod proxy_http

```
profesor@srv-daw:~$ sudo a2enmod proxy_http
Considering dependency proxy for proxy_http:
Module proxy already enabled
Enabling module proxy_http.
To activate the new configuration, you need to run:
  systemctl restart apache2
profesor@srv-daw:~$
```

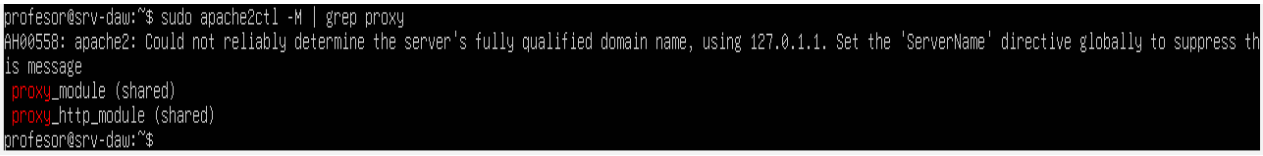
Comprobar:

sudo apache2ctl -M | grep proxy

Debería aparecer:

proxy_module

proxy_http_module



Actividad 2. Crear Virtual Host para Flask

Crear:

sudo nano /etc/apache2/sites-available/flask.conf

Contenido Apache Config:

<VirtualHost *:80>

ServerName flask.local

ProxyPreserveHost On

ProxyPass / http://127.0.0.1:5000/

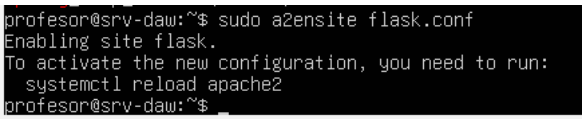
ProxyPassReverse / http://127.0.0.1:5000/

</VirtualHost>

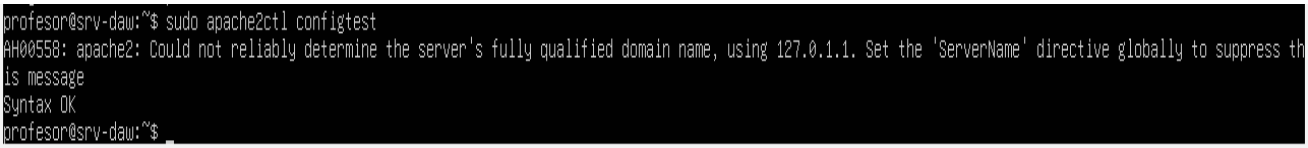
Apache escuchará por el puerto 80 y enviará las peticiones a 5000 donde se ejecuta Flask.

Actividad 3. Activar el sitio

sudo a2ensite flask.conf



Comprobar: **sudo apache2ctl configtest**



Resultado esperado: **Syntax OK**

Recargar Apache: **sudo systemctl reload apache2**

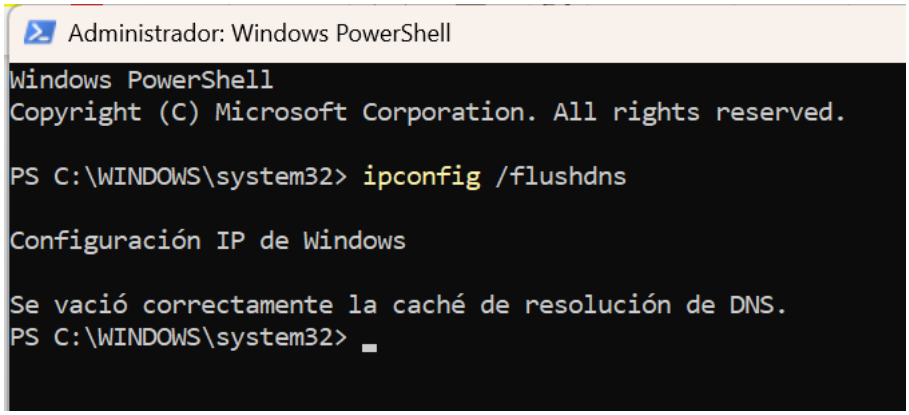
Actividad 4. Configurar resolución de nombres

En Windows: C:\Windows\System32\drivers\etc\hosts

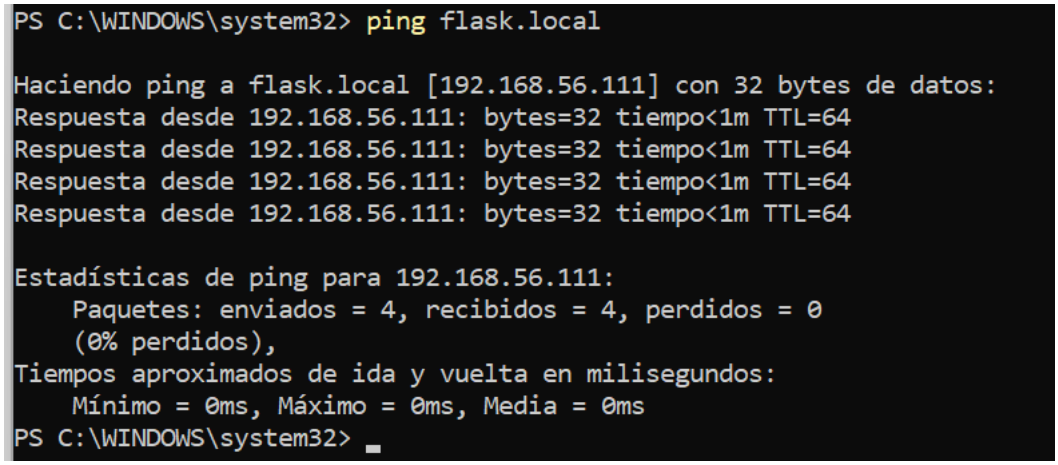
Añadir: **192.168.56.111 flask.local**

Guardar.

Vaciar caché DNS: **ipconfig /flushdns**



Comprobar: **ping flask.local**



Actividad 5. Mantener Flask ejecutándose

Terminal 1:

cd ~/flask

source venv/bin/activate

python app.py

```
(venv) profesor@srv-daw:~/flask$ ping flask.local
ping: flask.local: Temporary failure in name resolution
(venv) profesor@srv-daw:~/flask$ cd ~/flask
(venv) profesor@srv-daw:~/flask$ source venv/bin/activate
(venv) profesor@srv-daw:~/flask$ python app.py
 * Serving Flask app 'app'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://10.0.2.15:5000
Press CTRL+C to quit
```

Debe seguir mostrando: **Running on http://127.0.0.1:5000**

Actividad 6. Acceder mediante Apache

Desde Windows abrir: **http://flask.local**

Resultado esperado:

Aplicación Flask

Fecha actual:

...



¿Qué ha ocurrido?

Antes: Cliente → Flask

Ahora: Cliente → Apache → Flask

Actividad 7. Comprobación de puertos

Ejecutar: **ss -tulpn**

Deberían aparecer:

80

443

5000

```
(venv) profesor@srv-daw:~/flask$ ss -tulpn
Netid     State      Recv-Q     Send-Q                  Local Address:Port          Peer Address:Port     Process
udp       UNCONN     0          0                           127.0.0.1:323              0.0.0.0:*
udp       UNCONN     0          0                          127.0.0.54:53               0.0.0.0:*
udp       UNCONN     0          0                       127.0.0.53%lo:53               0.0.0.0:*
udp       UNCONN     0          0                192.168.56.111%enp0s8:68              0.0.0.0:*
udp       UNCONN     0          0                     10.0.2.15%enp0s3:68              0.0.0.0:*
udp       UNCONN     0          0                             [::1]:323                  [::]:*
tcp       LISTEN     0          4096                    127.0.0.53%lo:53               0.0.0.0:*
tcp       LISTEN     0          4096                       127.0.0.54:53               0.0.0.0:*
tcp       LISTEN     0          80                          127.0.0.1:3306             0.0.0.0:*
tcp       LISTEN     0          4096                          0.0.0.0:22               0.0.0.0:*
tcp       LISTEN     0          511                                 *:80                     *:*
tcp       LISTEN     0          4096                             [::]:22                  [::]:*
tcp       LISTEN     0          511                                 *:443                    *:*
(venv) profesor@srv-daw:~/flask$
```

```
profesor@srv-daw:~$ ss -tulpn
Netid         State          Recv-Q         Send-Q                  Local Address:Port          Peer Address:Port        Process
udp           UNCONN         0              0                           127.0.0.1:323              0.0.0.0:*
udp           UNCONN         0              0                          127.0.0.54:53               0.0.0.0:*
udp           UNCONN         0              0                       127.0.0.53%lo:53               0.0.0.0:*
udp           UNCONN         0              0                192.168.56.111%enp0s8:68              0.0.0.0:*
udp           UNCONN         0              0                     10.0.2.15%enp0s3:68              0.0.0.0:*
udp           UNCONN         0              0                             [::1]:323                  [::]:*
tcp           LISTEN         0              4096                    127.0.0.53%lo:53               0.0.0.0:*
tcp           LISTEN         0              4096                       127.0.0.54:53               0.0.0.0:*
tcp           LISTEN         0              80                          127.0.0.1:3306             0.0.0.0:*
tcp           LISTEN         0              4096                          0.0.0.0:22               0.0.0.0:*
tcp           LISTEN         0              511                                 *:80                     *:*
tcp           LISTEN         0              4096                             [::]:22                  [::]:*
tcp           LISTEN         0              511                                 *:443                    *:*
profesor@srv-daw:~$
```

¿Qué servicio escucha en cada puerto?

80 → Apache HTTP

443 → Apache HTTPS

La práctica está funcionando exactamente como debe. ¿Por qué no ves el puerto 5000 en el navegador? Porque ya hemos configurado correctamente el Reverse Proxy.

Cuando accedemos a http://flask.local el navegador habla con Apache en el Puerto 80. No habla directamente con Flask. Apache es quien internamente reenvía la petición hacia 127.0.0.1:5000 donde está ejecutándose Fl**ask.**

Navegador → flask.local → Puerto 80 → Apache → Flask → Puerto 5000

Prueba definitiva con Flask arrancado ejecuta curl http://127.0.0.1:5000

```
tcp            LISTEN          0                511
profesor@srv-daw:~$ curl http://127.0.0.1:5000

    <h1>Aplicación Flask</h1>
    <p>Fecha actual: 2026-10-20 13:14:17.880203</p>
    profesor@srv-daw:~$
```

Actividad 8. Comprobar con Nmap

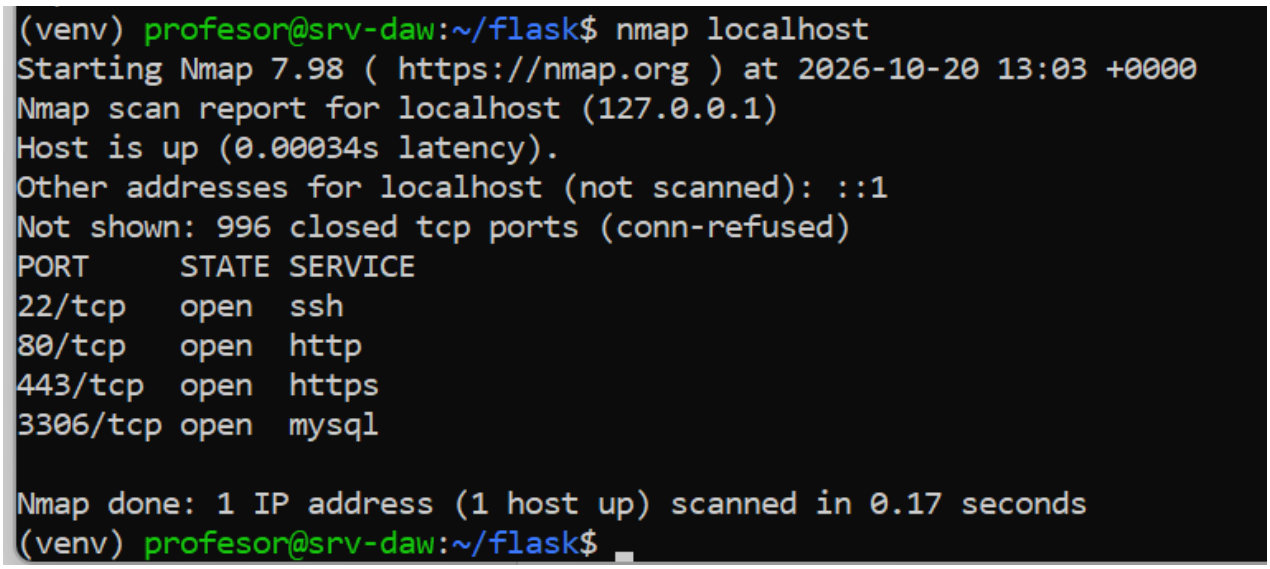
nmap localhost

Identificar:

22/tcp

80/tcp

443/tcp



Preguntas de reflexión

¿Por qué no es recomendable exponer Flask directamente a Internet?

Porque el servidor integrado de Flask está pensado para desarrollo y no ofrece el rendimiento, seguridad y escalabilidad de Apache o Nginx.

¿Qué ventaja aporta Apache como Reverse Proxy?

Permite gestionar HTTPS, certificados SSL/TLS, logs, control de acceso y balanceo de carga, mientras Flask se centra únicamente en ejecutar la aplicación.

¿Qué ocurriría si Flask se detiene?

Apache seguiría funcionando, pero devolvería errores al intentar reenviar las peticiones al puerto 5000.

Conclusión

Con esta práctica hemos desplegado una arquitectura profesional:

Windows → HTTPS → Apache → Reverse Proxy → Flask → Python 3.14

y cubierto completamente el criterio **RA3.c Se han desplegado aplicaciones Python** además de avanzar de forma muy importante en los otros criterios

- **RA3.a Instalación y configuración de Apache**
- **RA3.e Optimización de configuraciones para distintos entornos de explotación.**

Vamos a culminar ahora la U2 con:

DNS → HTTPS → Apache → Reverse Proxy → Flask → Python

Cubriendo completamente:

- **RA3.a Instalar y configurar servidores Apache.**
- **RA3.c Desplegar aplicaciones Python.**
- **RA3.e Optimizar configuraciones para distintos entornos.**
- Refuerzo de **RA2.c HTTP vs HTTPS.**

Objetivo final

Actualmente accedes mediante http://192.168.56.111:5000 o http://flask.local:5000



Queremos llegar a https://flask.local sin mostrar el puerto 5000.

Paso 1. Crear un VirtualHost para Flask

Crear: **sudo nano /etc/apache2/sites-available/flask.conf**

Contenido:

Apache Config

```
<VirtualHost *:80>

ServerName flask.local

ProxyPreserveHost On

ProxyPass / http://127.0.0.1:5000/

ProxyPassReverse / http://127.0.0.1:5000/
```

</VirtualHost>

Explicación

Apache escucha: 80 y reenvía automáticamente a: 5000 donde se ejecuta Flask. El usuario nunca ve Flask directamente.

Paso 2. Activar módulos proxy

Si aún no lo hemos hecho:

sudo a2enmod proxy

sudo a2enmod proxy_http

Verificar: **sudo apache2ctl -M | grep proxy**

Resultado esperado:

proxy_module

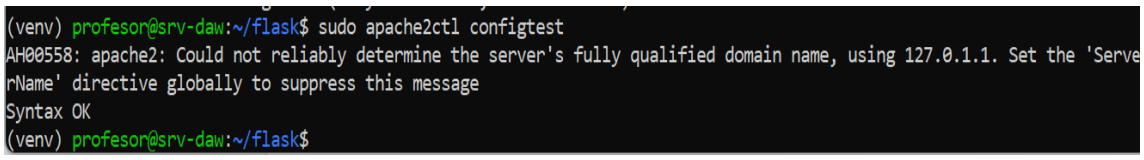
proxy_http_module

Paso 3. Activar el sitio

sudo a2ensite flask.conf



Comprobar: **sudo apache2ctl configtest**



Debe aparecer: **Syntax OK**

Recargar Apache: **sudo systemctl reload apache2**

Paso 4. Configurar DNS local

Editar en Windows: C:\Windows\System32\drivers\etc\hosts

Añadir: **192.168.56.111 flask.local**

Vaciar caché DNS: **ipconfig /flushdns**

```
PS C:\WINDOWS\system32> ipconfig /flushdns

Configuración IP de Windows

Se vació correctamente la caché de resolución de DNS.
PS C:\WINDOWS\system32>
```

Comprobar: **ping flask.local**

```
PS C:\WINDOWS\system32> ping flask.local

Haciendo ping a flask.local [192.168.56.111] con 32 bytes de datos:
Respuesta desde 192.168.56.111: bytes=32 tiempo=1ms TTL=64
Respuesta desde 192.168.56.111: bytes=32 tiempo<1m TTL=64
Respuesta desde 192.168.56.111: bytes=32 tiempo<1m TTL=64
Respuesta desde 192.168.56.111: bytes=32 tiempo<1m TTL=64

Estadísticas de ping para 192.168.56.111:
    Paquetes: enviados = 4, recibidos = 4, perdidos = 0
    (0% perdidos),
Tiempos aproximados de ida y vuelta en milisegundos:
    Mínimo = 0ms, Máximo = 1ms, Media = 0ms
PS C:\WINDOWS\system32>
```

Debe resolver: **192.168.56.111**

Paso 5. Verificar Reverse Proxy

Mantén Flask ejecutándose:

source ~/flask/venv/bin/activate

python app.py

```
(venv) profesor@srv-daw:~/flask$ source ~/flask/venv/bin/activate
(venv) profesor@srv-daw:~/flask$ python app.py
 * Serving Flask app 'app'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://10.0.2.15:5000
Press CTRL+C to quit
```

Otra terminal:

sudo systemctl status apache2

```
profesor@srv-daw:sudo systemctl status apache2
[sudo: authenticate] Password:
● apache2.service - The Apache HTTP Server
     Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset: enabled)
     Active: active (running) since Tue 2026-10-20 12:42:53 UTC; 40min ago
 Invocation: 1a6032a0e7a14553a2703edb9fc3593c
       Docs: https://httpd.apache.org/docs/2.4/
    Process: 4424 ExecReload=/usr/sbin/apachectl graceful (code=exited, status=0/SUCCESS)
   Main PID: 3828 (apache2)
     Status: "Total requests: 6; Idle/Busy workers 100/0;Requests/sec: 0.0025; Bytes served/sec:   2 B/sec"
      Tasks: 6 (limit: 1715)
     Memory: 13M (peak: 26.4M)
        CPU: 1.968s
     CGroup: /system.slice/apache2.service
             ├─3828 /usr/sbin/apache2 -k start -DFOREGROUND
             ├─4433 /usr/sbin/apache2 -k start -DFOREGROUND
             ├─4434 /usr/sbin/apache2 -k start -DFOREGROUND
             ├─4435 /usr/sbin/apache2 -k start -DFOREGROUND
             ├─4436 /usr/sbin/apache2 -k start -DFOREGROUND
             └─4437 /usr/sbin/apache2 -k start -DFOREGROUND

oct 20 12:42:53 srv-daw systemd[1]: Starting apache2.service - The Apache HTTP Server...
oct 20 12:42:53 srv-daw apachectl[3828]: AH00558: apache2: Could not reliably determine the server's fully qualified domain name, using 127.0.1.1. Set the 'Ser>
oct 20 12:42:53 srv-daw systemd[1]: Started apache2.service - The Apache HTTP Server.
oct 20 12:51:33 srv-daw systemd[1]: Reloading apache2.service - The Apache HTTP Server...
oct 20 12:51:34 srv-daw systemd[1]: Reloaded apache2.service - The Apache HTTP Server.
oct 20 12:51:34 srv-daw apachectl[4019]: AH00558: apache2: Could not reliably determine the server's fully qualified domain name, using 127.0.1.1. Set the 'Ser>
oct 20 13:19:46 srv-daw systemd[1]: Reloading apache2.service - The Apache HTTP Server...
oct 20 13:19:46 srv-daw apachectl[4424]: AH00558: apache2: Could not reliably determine the server's fully qualified domain name, using 127.0.1.1. Set the 'Ser>
oct 20 13:19:46 srv-daw systemd[1]: Reloaded apache2.service - The Apache HTTP Server.
lines 1-28/28 (END)
```

Abrir: **http://flask.local**

Si aparece:

Aplicación Flask

Fecha actual...

Apache está reenviando correctamente al puerto 5000.



¿Cómo comprobar que Apache reenvía al puerto 5000?

Ejecuta: **ss -tulpn**

Verás:

80

443

```
profesor@srv-daw:~$ ss -tulpn
Netid       State        Recv-Q       Send-Q                Local Address:Port           Peer Address:Port      Process
udp         UNCONN       0            0                        127.0.0.1:323                 0.0.0.0:*
udp         UNCONN       0            0                     127.0.0.54:53                    0.0.0.0:*
udp         UNCONN       0            0                  127.0.0.53%lo:53                    0.0.0.0:*
udp         UNCONN       0            0              192.168.56.111%enp0s8:68                0.0.0.0:*
udp         UNCONN       0            0                  10.0.2.15%enp0s3:68                 0.0.0.0:*
udp         UNCONN       0            0                          [::1]:323                    [::]:*
tcp         LISTEN       0            4096               127.0.0.53%lo:53                    0.0.0.0:*
tcp         LISTEN       0            4096                  127.0.0.54:53                    0.0.0.0:*
tcp         LISTEN       0            80                    127.0.0.1:3306                  0.0.0.0:*
tcp         LISTEN       0            128                     0.0.0.0:5000                  0.0.0.0:*          users:(("python",pid=4482,fd=3))
tcp         LISTEN       0            4096                    0.0.0.0:22                    0.0.0.0:*
tcp         LISTEN       0            511                           *:80                          *:*
tcp         LISTEN       0            4096                       [::]:22                     [::]:*
tcp         LISTEN       0            511                           *:443                         *:*
profesor@srv-daw:~$
```

Interpretación:

80 → Apache HTTP

443 → Apache HTTPS

También puedes detener Flask: **CTRL + C**

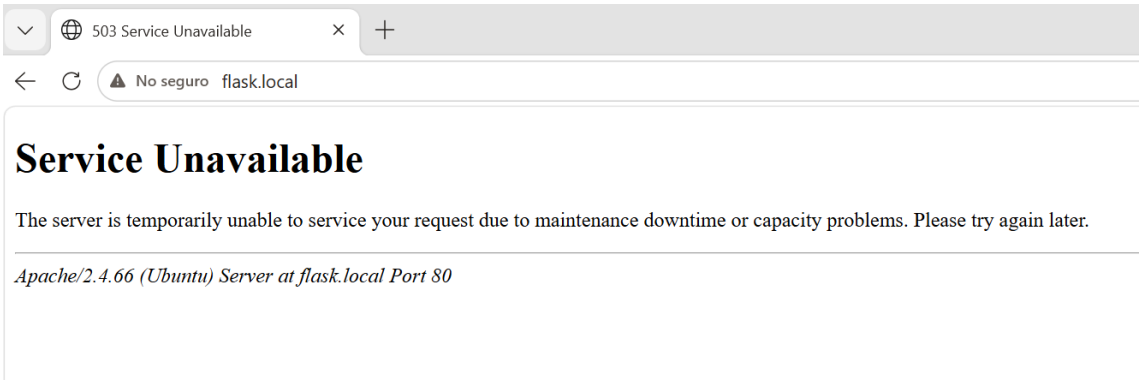
y volver a abrir: **http://flask.local**

Obtendrás un error:

502

503

Proxy Error



Eso demuestra que Apache depende del servicio Flask en el puerto 5000.

Después vuelve a arrancarlo: **python app.py**



Paso 6. Añadir HTTPS al Reverse Proxy

Crear: **sudo nano /etc/apache2/sites-available/flask-ssl.conf**

Contenido Apache Config:

<VirtualHost *:443>

ServerName flask.local

SSLEngine on

SSLCertificateFile /etc/apache2/ssl/empresa1.crt

SSLCertificateKeyFile /etc/apache2/ssl/empresa1.key

ProxyPreserveHost On

ProxyPass / http://127.0.0.1:5000/

ProxyPassReverse / http://127.0.0.1:5000/

</VirtualHost>

Activar el sitio HTTPS: **sudo a2ensite flask-ssl.conf**

Comprobar: **sudo apache2ctl configtest**

```
(venv) profesor@srv-daw:~/flask$ sudo a2ensite flask-ssl.conf
Enabling site flask-ssl.
To activate the new configuration, you need to run:
  systemctl reload apache2
(venv) profesor@srv-daw:~/flask$ sudo apache2ctl configtest
AH00558: apache2: Could not reliably determine the server's fully qualified domain name, using 127.0.1.1. Set the 'Serve
rName' directive globally to suppress this message
Syntax OK
(venv) profesor@srv-daw:~/flask$
```

Recargar: **sudo systemctl reload apache2**

Después vuelve a arrancar: **python app.py**

Verificación

Abrir: https://flask.local



Arquitectura final obtenida

Cliente (Windows) → HTTPS (443) →Apache → Reverse Proxy → Flask → Python

Preguntas de reflexión

¿Por qué no accedemos directamente a Flask en producción?

Porque el servidor integrado de Flask está pensado para desarrollo, mientras que Apache aporta seguridad, gestión de HTTPS, logs y mayor rendimiento.

¿Qué ventajas aporta el Reverse Proxy?

- Oculta Flask al exterior.
- Centraliza HTTPS.
- Permite balanceo de carga en el futuro.
- Facilita la administración.
- Mejora la seguridad.

¿Qué ocurriría si Flask se detiene?

Apache seguirá escuchando en los puertos 80 y 443, pero no podrá entregar contenido porque no podrá contactar con la aplicación Python en el puerto 5000.

Resultado de aprendizaje cubiertos

- **RA3.a** Instalación y configuración avanzada de Apache.
- **RA3.c** Despliegue de aplicaciones Python.
- **RA3.e** Optimización de arquitecturas web mediante Reverse Proxy.
- Refuerzo de **RA2.c** HTTP vs HTTPS.

Con esto hemos cubierto todos los criterios principales de la UD2.